

CS6493 NLP Group Project Progress Report

Topic 1: Mathematical Reasoning Ability of Large Language Models

April 22, 2026

Student Name: Chen Baizheng

Student ID: 72510800

1 Project Positioning and Goal

This project studies mathematical reasoning of small open-weight LLMs under prompt-based strategies, with focus on two dimensions: (1) answer correctness and (2) reasoning efficiency.

The second dimension is our main extension beyond standard assignment requirements. We aim to build an evaluation protocol that compares methods not only by accuracy but also by reasoning quality and stopping behavior.

Our plan follows three principles:

1. assignment compliance first,
2. controlled innovation second,
3. behavior-aware evaluation third.

2 Experimental Scope

2.1 Models

- Qwen/Qwen2.5-Math-1.5B-Instruct
- deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B

2.2 Datasets

- HuggingFaceH4/MATH-500
- openai/gsm8k
- HuggingFaceH4/aime_2024

The current controlled setting uses 30 sampled items per dataset for progress-stage validation, with planned expansion in final runs.

2.3 Method Set

Required methods:

- CoT zero-shot
- CoT few-shot
- Self-Refine
- Self-Consistency ($n = 5$)

Extended methods:

- TIR (Tool-Integrated Reasoning)
- SR-SD-TIR (plan $\rightarrow$ solve-with-tools $\rightarrow$ reflect)

3 Method Design Rationale

3.1 Baseline methods: what each one tests

- **CoT zero-shot**: reference baseline without demonstrations.
- **CoT few-shot**: tests whether in-context exemplars improve structure and stability.
- **Self-Refine**: tests whether second-pass self-critique corrects first-pass errors.
- **Self-Consistency**: tests robustness gains from multi-sample voting.

These methods form a progression from single-pass reasoning to multi-pass and multi-sample aggregation.

3.2 Extension design: why TIR and SR-SD-TIR

- **TIR**: introduces executable code plus execution feedback for arithmetic/symbolic verification.
- **SR-SD-TIR**: introduces staged control (planning, solving, reflecting) to reduce wasted reasoning trajectories.

Our core hypothesis is that staged and tool-assisted methods can improve answer-efficiency behavior under the same model-size budget.

3.3 Comparison logic

Methods are compared under a shared protocol using correctness, response budget, answer-emergence signals, and over-reasoning indicators, rather than a single headline metric only.

4 Evaluation Metric Design (Key Focus)

4.1 Core metrics (assignment-aligned)

- Accuracy
- Response length (tokens/chars)

4.2 Behavior-aware metrics (our extension)

- Reflection count
- Answer count
- First answer token index
- Tail ratio (tokens generated after first answer / total tokens)

Interpretation: high answer count combined with high tail ratio often indicates repeated or unnecessary post-answer reasoning.

4.3 Composite score design in $[0, 1]$

Per-sample score:

$$\text{score}_i = c \cdot \text{answer_factor}^{0.3} \cdot \text{length_factor}^{0.4} \cdot \text{reflection_factor}^{0.3}$$

where $c \in \{0, 1\}$ is a correctness gate and final score is clipped to $[0, 1]$.

Factor design:

- **length_factor**: penalizes excessive length beyond reference budget.
- **answer_factor** = $\text{count_factor} \times \text{tail_factor}$:
 - **count_factor** penalizes excessive answer repetitions beyond target a^* ,
 - **tail_factor** penalizes excessive continuation after first answer.
- **reflection_factor**: encourages moderate reflection over both no-reflection and over-reflection.

Current default parameters:

- weights: $\text{answer} = 0.3$, $\text{length} = 0.4$, $\text{reflection} = 0.3$

- $a^* = 2, \tau_a = 2$
- $r^* = 2, \tau_r = 2$
- $\rho^* = 0.25, \tau_{tail} = 0.20$

4.4 Planned statistical reporting

For each model-dataset-method group, we report:

- mean/median/mode of composite scores,
- single-metric means,
- per-question records for error analysis.

This two-level reporting supports macro comparison and qualitative diagnosis simultaneously.

5 Reproducibility and Fairness Controls

To ensure fair comparison:

1. shared datasets and fixed sample files,
2. consistent prompt-method interface,
3. YAML-managed method configurations,
4. shared evaluation and scoring protocol.

Question consistency is guaranteed by fixed sampled JSONL files. As long as these files are unchanged, compared runs use identical question sets.

6 Progress Status

At this stage, the research protocol and metric framework are finalized. The remaining major workload is empirical execution at scale and final analysis writing.

6.1 Completed

- method portfolio definition and comparison protocol,
- metric and composite-score design,
- evaluation output schema and summary strategy,
- run configuration structure for controlled experiments.

6.2 Ongoing / Next

- full matrix execution (model $\times$ dataset $\times$ method),
- sensitivity checks for key score parameters,
- final comparative discussion with complete quantitative evidence.

7 Next-Step Commitments

Before final submission:

1. complete required-method full runs,
2. run extension methods under matched conditions,
3. deliver full quantitative tables and interpretation,
4. conclude with trade-off analysis among correctness, efficiency, and reasoning behavior.

8 References

1. Wei, J., Wang, X., Schuurmans, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*.
2. Wang, X., Wei, J., Schuurmans, D., et al. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv:2203.11171*.
3. Madaan, A., Tandon, N., Gupta, P., et al. (2023/2024). Self-Refine: Iterative refinement with self-feedback. *NeurIPS*.
4. Zhou, D., Scharli, N., Hou, L., et al. (2022). Least-to-Most prompting enables complex reasoning in large language models. *arXiv:2205.10625*.
5. Zhang, Z., Zhang, A., Li, M., Smola, A. (2022). Automatic chain of thought prompting in large language models. *arXiv:2210.03493*.
6. Gou, Z., Shao, Z., Gong, Y., et al. (2023). ToRA: A tool-integrated reasoning agent for mathematical problem solving. *arXiv:2309.17452*.
7. Shao, Z., Wang, P., Zhu, Q., et al. (2024). DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint*.
8. Cobbe, K., Kosaraju, V., Bavarian, M., et al. (2021). Training verifiers to solve math word problems. *arXiv:2110.14168*.
9. Cobbe, K., et al. (2021). GSM8K: Grade school math 8K benchmark. *arXiv:2110.14168*.

10. Hendrycks, D., et al. (2021). Measuring mathematical problem solving with the MATH dataset. *NeurIPS*.
11. Lightman, H., Kosaraju, V., Burda, Y., et al. (2023). Let’s verify step by step. *arXiv:2305.20050*.
12. Shinn, N., Cassano, F., Berman, E., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint*.
13. Zhou, P., Madaan, A., Potharaju, S. P., et al. (2024). Self-Discover: Large language models self-compose reasoning structures. *arXiv preprint*.
14. Team, HuggingFaceH4. (2024). MATH-500 dataset release.
15. Team, HuggingFaceH4. (2024). AIME 2024 dataset release.